

1.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Load dataset
data = pd.read_csv("spam.csv")

# Features and target
X = data['Message']
y = data['Category']

# Convert text to numerical features
cv = CountVectorizer()
X = cv.fit_transform(X)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Train model
model = MultinomialNB()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

Output:

```

- RESTART: C:/Users/amaci/AppData/Local/Programs/Python/Python313/6istribution.p
7
First 5 Rows:
Category      Message
0      ham      Go until jurong point, crazy.. Available only ...
1      ham      Ok lar... Joking wif u oni...
2      spam      Free entry in 2 a wkly comp to win FA Cup fina...
3      ham      U dun say so early hor... U s already then say...
4      ham      Nah I don't think he goes to usf, he lives aro...

Accuracy: 0.9854592282152466

Confusion Matrix:
[[957   8]
 [  7 142]]

Classification Report:
              precision    recall  F1-score   support

      0       0.99       0.98       0.99       966
      1       0.98       0.95       0.95       149

 accuracy      0.99
 macro avg     0.97       0.93       0.97       1115
weighted avg     0.99       0.98       0.99       1115

Predictions [1]
Probabilities
[[[6.70756246e-88  9.99999932e-81]]]

This email is SPAM.

```

2.

Code:

```
import pandas as pd
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.tree import DecisionTreeClassifier
```

```
data = pd.read_csv("student.csv")
```

```
X = data[['StudyHours','Attendance','PreviousMarks']]
y = data['Performance']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
model = DecisionTreeClassifier()
```

```
model.fit(X_train, y_train)
```

```
print(model.predict(X_test))
```

Output:

```

First 5 Rows:
 school sex  age address famsize Pstatus  ...  Walc  health  absences  G1  G2  G3
0      GP  F   18      U    GT3      A  ...    1      3         6  5  6  6
1      GP  F   17      U    GT3      T  ...    1      3         4  5  5  6
2      GP  F   15      U    LE3      T  ...    3      3        10  7  8  10
3      GP  F   15      U    GT3      T  ...    1      5         2  15 14  15
4      GP  F   16      U    GT3      T  ...    2      5         4  6  10 10

[5 rows x 33 columns]

Dataset Shape: (395, 33)

Model Performance
Mean Absolute Error (MAE): 1.139240506329114
Mean Squared Error (MSE): 4.2025316455696204
R2 Score: 0.795048916950583

Prediction for First Student
Predicted Final Grade (G3): 6.0
Actual Final Grade (G3): 6

```

3.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

```
data = pd.read_csv("weather.csv")
```

```
X = data[['Temperature','Humidity','WindSpeed']]
```

```
y = data['Weather']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

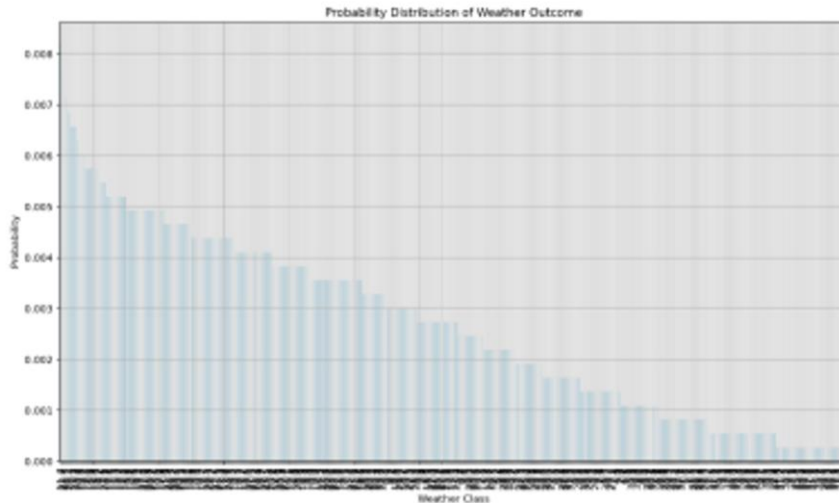
```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
print(model.predict(X_test))
```

Output:

[illegible]



4.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv("fraud.csv")

le = LabelEncoder()
data['Type'] = le.fit_transform(data['Type'])

X = data[['Amount', 'Type']]
y = data['Fraud']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Output:

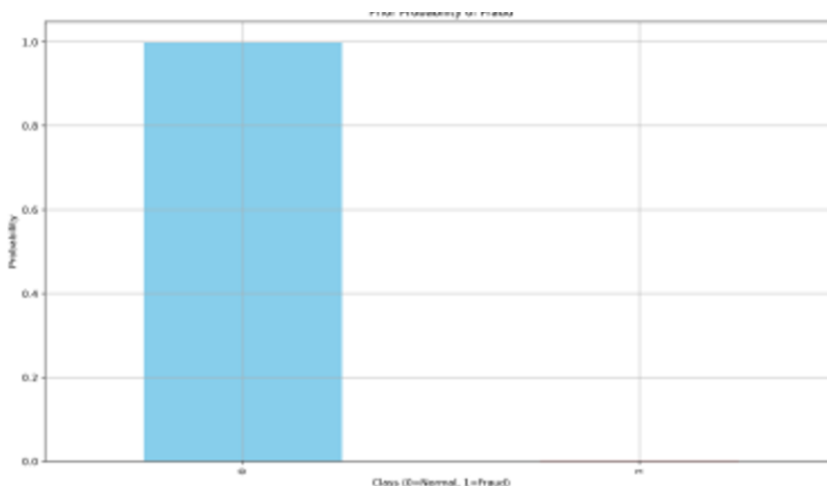
```
Python 3.11.2 Shell [C:\Users\user\AppData\Local\Programs\Python\Python311\python.exe]
>>>
= RESULTS: c:\Users\user\AppData\Local\Programs\Python\Python311\distribution.py
===== RESULTS =====
Time    S1    V0    V1    ...    S21    V20    Recall    Class
0  0.0  -0.358887  -0.072780  2.338387  ...  -0.133558  -0.331053  0.09,02  0
1  0.0  -1.195057  -0.246150  -0.166480  ...  -0.000993  -0.314724  2.69  0
2  1.0  -1.358358  -1.140163  1.773290  ...  -0.005353  -0.357552  376.64  0
3  1.0  -0.948272  -0.189226  1.792993  ...  -0.062723  -0.265468  123.93  0
4  1.0  -0.158933  -0.077737  1.348718  ...  -0.359433  -0.215153  49.94  0

[5 rows x 10 columns]

Dataset shape: (10000, 11)

Columns:
Index(['Time', 'S1', 'S2', 'S3', 'S4', 'S5', 'S6', 'S7', 'S8', 'S9', 'S10',
       'S11', 'S12', 'S13', 'S14', 'S15', 'S16', 'S17', 'S18', 'S19', 'S20',
       'S21', 'S22', 'S23', 'S24', 'S25', 'S26', 'S27', 'S28', 'S29', 'Recall',
       'Class'],
      dtype='object')

===== PRIOR PROBABILITY =====
Class
0  0.948272
1  0.051727
Name: proportion, dtype: float64
```



5.

Code:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = pd.read_csv("fraud.csv")

le = LabelEncoder()
```

```
data['Type'] = le.fit_transform(data['Type'])
```

```
X = data[['Amount', 'Type']]
```

```
y = data['Fraud']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
```

```
model = GaussianNB()
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print(classification_report(y_test, y_pred))
```

Output:

```
===== DATASET =====
   Age  Sex  Chest pain type  ...  Number of vessels fluro  Thallium  Heart Disease
0   70    1             4  ...              3              3      Presence
1   67    0             3  ...              0              7      Absence
2   57    1             2  ...              0              7      Presence
3   64    1             4  ...              1              7      Absence
4   74    0             2  ...              1              3      Absence

[5 rows x 14 columns]

Dataset Shape: (270, 14)

Column Names:
Index(['Age', 'Sex', 'Chest pain type', 'BP', 'Cholesterol', 'FBS over 120',
       'EKG results', 'Max HR', 'Exercise angina', 'ST depression',
       'Slope of ST', 'Number of vessels fluro', 'Thallium', 'Heart Disease'],
      dtype='str')

===== PRIOR PROBABILITY =====
Heart Disease
Absence    0.555556
Presence   0.444444
Name: proportion, dtype: float64
```

